

UNIVERSIDAD LA SALLE BAJIO

KAGASTIAN 5000

Robot caminante inspirado en un pinguino

Proyecto Tercer Parcial - Programacion Avanzada

Programacion Avanzada

Karla Daniela Martínez Esparza

Integrantes

6100090 - Diego De la O Arellano

6100093 - Hazel Ramirez Vazquez

6100103 - Diego Sebastian Espinoza Perez

6100116 - Emilio Giovanni Meza Mendez

6100130 - David Alexander Torres Jalomo

Resumen

El Kagastian 5000 es un robot móvil tipo animal cuyo objetivo principal es desplazarse sin ruedas mediante un mecanismo de patas inspirado en la caminata y el balanceo lateral de un pinguino. El proyecto busca resolver el reto de coordinar programación, electrónica y mecánica en un prototipo capaz de caminar, detectar obstáculos y responder a instrucciones enviadas desde una computadora.

El sistema utiliza un Arduino UNO, un motorreductor controlado mediante un puente H L293D, un sensor ultrasonico HC-SR04 y un buzzer. La interfaz de usuario fue desarrollada en Python con Tkinter y se comunica con Arduino por Serial USB a 9600 baudios mediante PySerial. El control es cableado por USB, no inalámbrico: permite seleccionar modos de movimiento, detener el robot, ejecutar una prueba corta, ajustar la distancia de frenado y observar la telemetría enviada por el microcontrolador.

1. Introduccion

Los robots móviles tipo animal permiten estudiar problemas que no aparecen en un vehículo convencional: equilibrio, apoyo alternado, conversión de movimiento rotatorio en pasos y coordinación entre sensores y actuadores. Se eligió un pinguino porque su desplazamiento terrestre es reconocible por sus pasos cortos y su balanceo lateral, rasgos que podían representarse mediante un mecanismo sencillo.

Motivacion

La motivación fue aplicar programación orientada a objetos y principios SOLID en un sistema físico real. La separación del software en clases permitió adaptar el proyecto cuando la configuración inicial de dos motores presentó diferencias de velocidad, una falla de motor y problemas asociados al L293D.

Alcance

El sistema controla un motorreductor, mide distancia, genera alertas sonoras, recibe comandos desde una interfaz gráfica y transmite mensajes de estado. Incluye modos automático, caminata continua, detenido y prueba corta. También conserva código y documentos suficientes para reproducir las conexiones y operar el robot.

Limitaciones

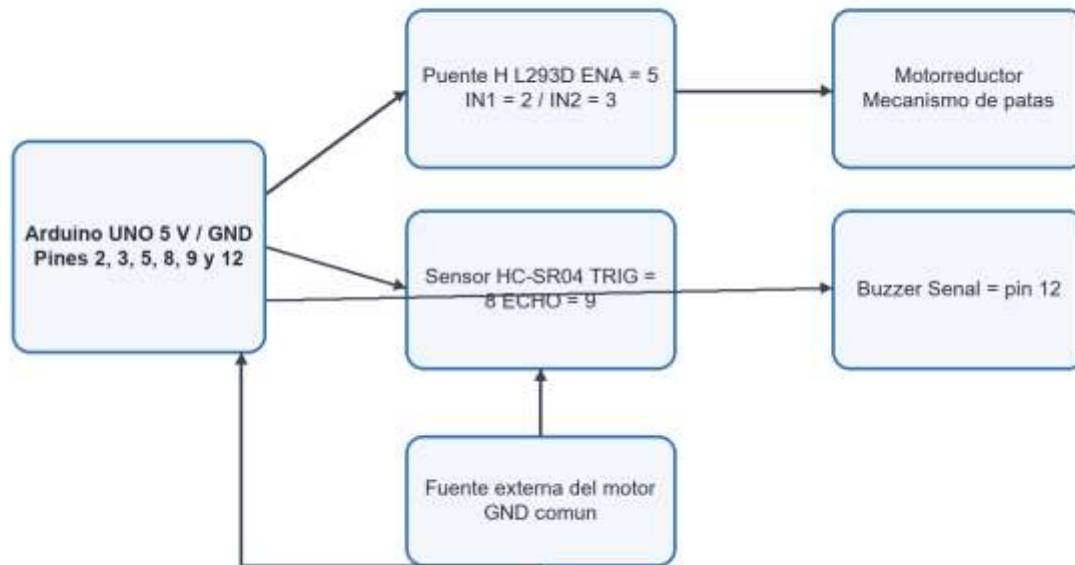
- La versión final de software controla un motor funcional; el montaje fotografiado muestra pruebas previas con dos motores.
- La comunicación depende del cable USB y del puerto COM; no se implementaron Bluetooth ni Wi-Fi.
- Se genera una desviación y tambaleo por alineación, peso, fricción y holgura mecánica.

2. Descripcion del sistema

Componentes de hardware

Componente	Funcion
Arduino UNO	Microcontrolador principal; ejecuta la logica y la comunicacion Serial.
HC-SR04	Sensor ultrasonico para medir la distancia frontal.
Motorreductor TT	Actuador que impulsa el mecanismo de patas.
L293D	Puente H que permite avance, retroceso, detencion y control PWM.
Buzzer	Alerta de inicio, cambio de modo y deteccion de obstaculos.
Alimentacion	Arduino por USB o power bank; motor mediante fuente externa con GND comun.

Diagrama general de conexiones

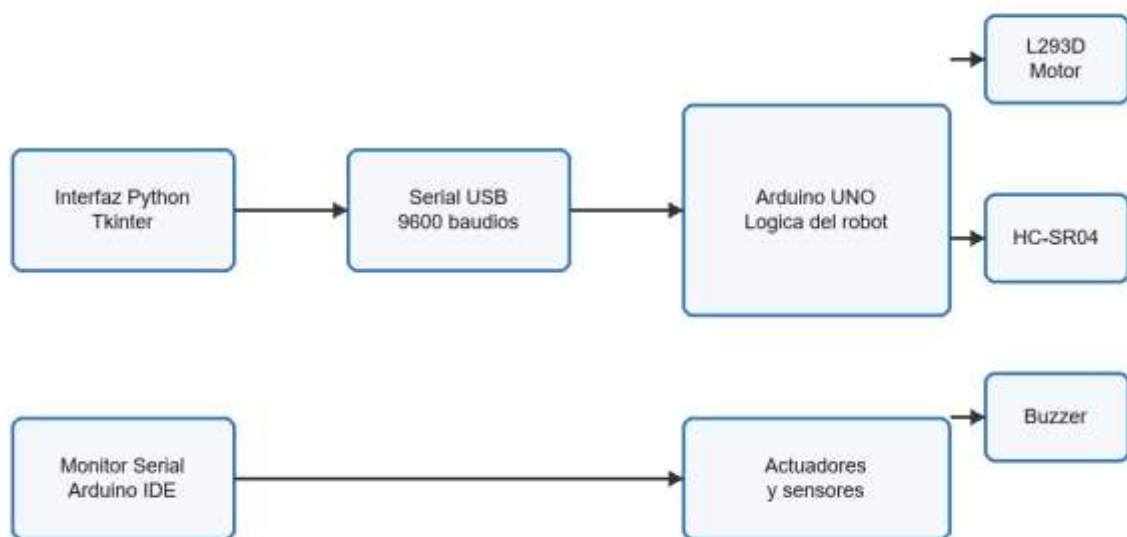


Arduino, L293D y fuente del motor deben compartir tierra (GND).

Componentes de software

Tecnologia	Uso en el proyecto
Arduino C/C++	Firmware del microcontrolador y clases de control.
Python 3	Interfaz de escritorio y comunicacion con el puerto serial.
Arduino IDE	Compilacion y carga del sketch main.ino.
Tkinter	Framework de interfaz grafica incluido con Python.
PySerial	Apertura, escritura y lectura no bloqueante del puerto COM.

Diagrama de arquitectura

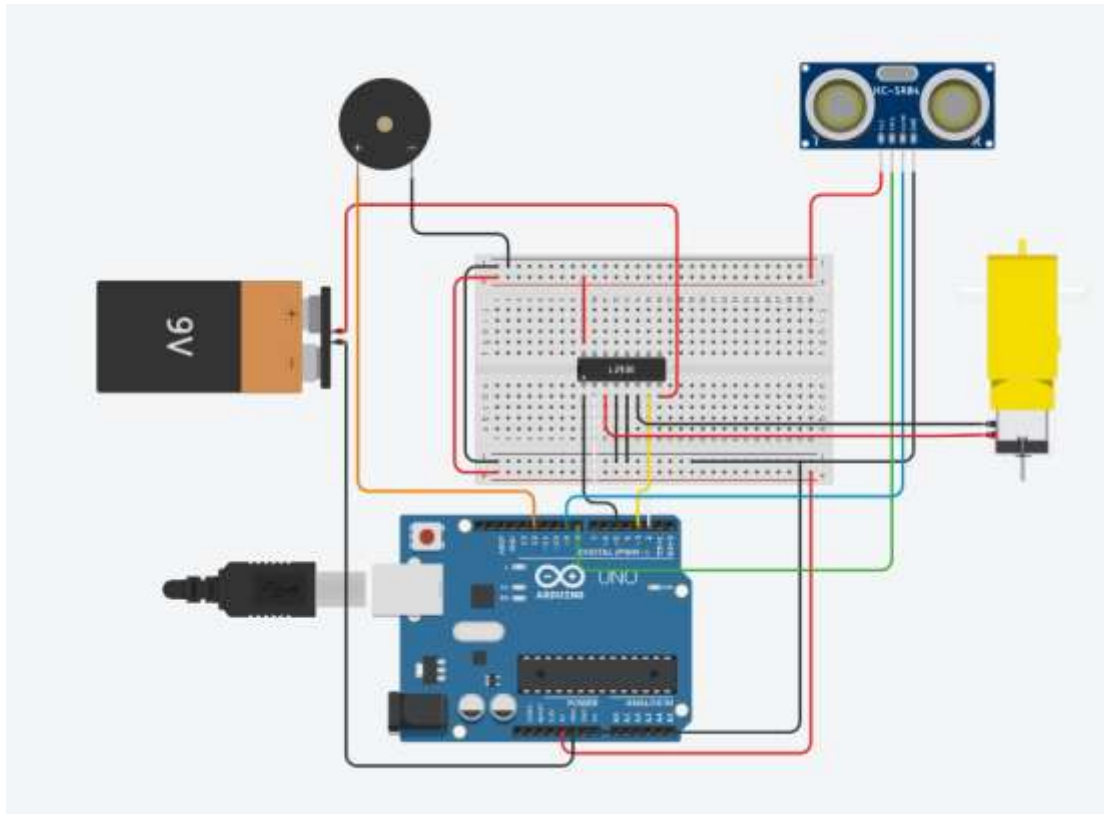


3. Arquitectura del programa

La arquitectura separa la presentacion, la comunicacion, la logica del robot y el acceso al hardware. La interfaz envia comandos y muestra telemetria; Arduino interpreta los comandos; PenguinRobot decide la accion; y las implementaciones concretas controlan el motor, el sensor y la alerta.

Arquitectura por capas





Clases e interfaces

Clase o interfaz	Responsabilidad
IMotor	Contrato pequeno para iniciar, definir velocidad, avanzar, retroceder y detener.
IDistanceSensor	Contrato para iniciar y obtener una medicion en centimetros.
IAlert	Contrato para sonidos de inicio, obstaculo y cambio de modo.
MotorL293D	Implementacion de IMotor para el puente H L293D.
UltrasonicSensor	Implementacion de IDistanceSensor para HC-SR04.
BuzzerAlert	Implementacion de IAlert mediante tone() y noTone().
PenguinRobot	Coordina modos, comandos, distancia minima y respuesta ante obstaculos.

Principios SOLID

Principio	Aplicacion
SRP	Cada modulo posee una responsabilidad: motor, sensor, alerta, coordinacion o presentacion.
OCP	Es posible agregar otro motor, sensor o alerta mediante una nueva clase sin reescribir PenguinRobot.
LSP	MotorL293D, UltrasonicSensor y BuzzerAlert pueden sustituirse por cualquier implementacion valida de sus interfaces.
ISP	Se usan tres interfaces pequenas; ninguna clase esta obligada a implementar operaciones que no necesita.
DIP	PenguinRobot recibe referencias a IMotor, IDistanceSensor e IAlert y depende de abstracciones.

4. Comunicacion computador-microcontrolador

La computadora y el Arduino se comunican por el mismo cable USB utilizado para programar la placa. La interfaz abre el puerto COM a 9600 baudios. Cada boton envia un caracter ASCII y el Arduino cambia su estado interno. En sentido contrario, Arduino envia lineas de texto terminadas en salto de linea; la GUI las consulta cada 100 ms sin bloquear el ciclo de eventos de Tkinter.

Dato	Comando	Accion
A	Automatico	Camina, mide distancia y evita obstaculos.
C	Caminata continua	Mantiene el motor avanzando para probar el mecanismo.
S	Detener	Apaga el motor.
D	Prueba corta	Avanza, pausa, retrocede, pita y queda detenido.
+	Frenar mas lejos	Aumenta 1 cm el umbral, con limite de 50 cm.
-	Frenar mas cerca	Disminuye 1 cm el umbral, con limite de 5 cm.

Datos recibidos por la computadora: mensajes de modo, confirmacion de cambios, avisos de obstaculo y mediciones con el formato "Distancia: N cm". La interfaz conserva cada linea en un registro de eventos y actualiza el indicador principal cuando identifica una medicion.

5. Interfaz grafica de usuario

La ventana principal concentra la conexion, el control y el diagnostico. El campo de puerto permite escribir un COM manualmente; Buscar puertos enumera los dispositivos disponibles; Conectar abre PySerial y Desconectar libera el recurso. El indicador rojo o verde informa el estado de conexion.

El indicador de distancia muestra la ultima lectura recibida. Los seis botones centrales traducen acciones del usuario a los caracteres del protocolo. El registro inferior conserva comandos enviados, respuestas de Arduino, errores de lectura y cambios de conexion, facilitando la deteccion de fallas.

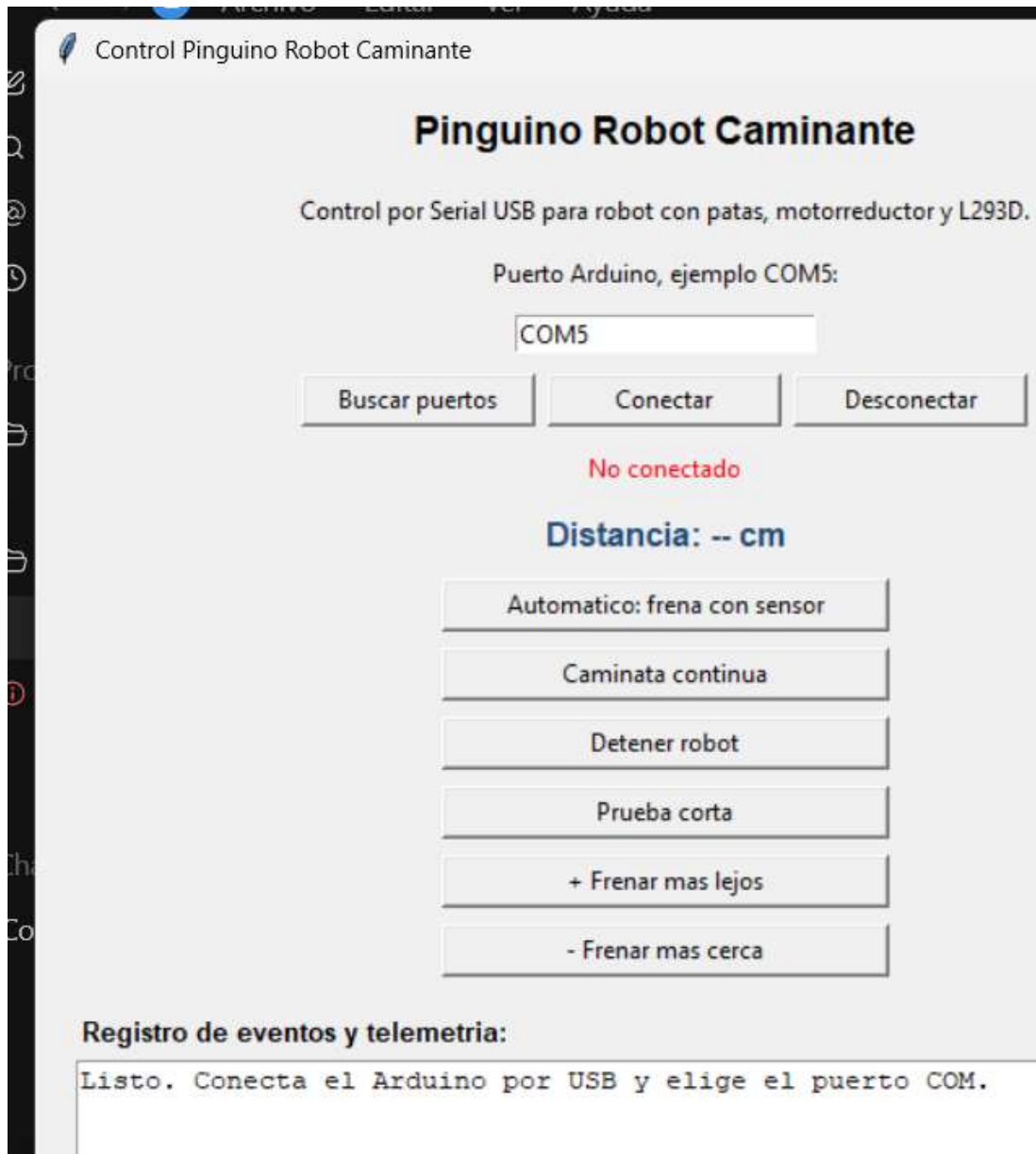


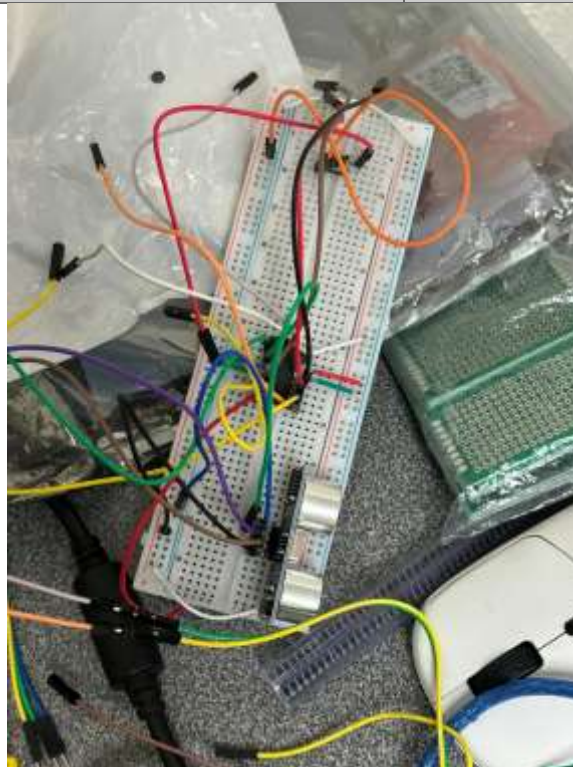
Figura 4. Interfaz actualizada: conexion, indicador de distancia, botones de control y registro de telemetria.

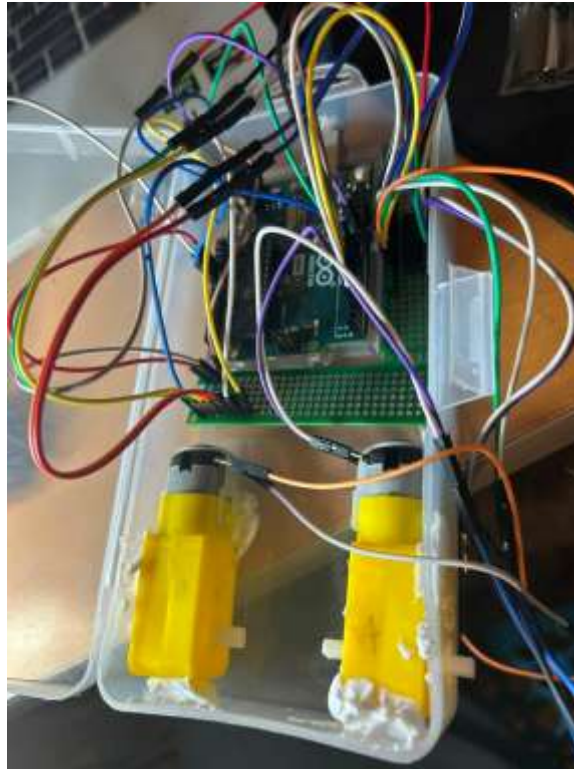
Elemento	Logica
1. Conexion	Selecciona el puerto y abre o cierra la comunicacion.
2. Estado	Muestra si la computadora esta conectada al Arduino.
3. Distancia	Refleja la telemetria "Distancia: N cm" recibida.
4. Movimiento	Selecciona automatico, continuo, detenido o prueba corta.
5. Umbral	Ajusta la distancia de frenado entre 5 y 50 cm.
6. Registro	Documenta eventos y errores de la sesion.

6. Resultados y experimentos

Las pruebas se ejecutaron de forma incremental. Primero se verificaron conexiones y actuadores por separado; despues se probó el protocolo serial, la respuesta al sensor y finalmente la locomocion integrada.

Experimento	Procedimiento	Resultado
Comunicacion Serial	Conectar Arduino por USB, abrir el COM a 9600 baudios y enviar A, C, S, D, + y -.	El firmware reconoce los comandos y responde con mensajes de modo o configuracion.
Sensor y alerta	Colocar un obstaculo frente al HC-SR04 en modo automatico y acercarlo al umbral.	El sistema reporta distancia, detiene el motor, activa el buzzer y retrocede brevemente.
Alimentacion	Comparar una pila rectangular de 9 V con una fuente externa para el motor.	La pila fue insuficiente; la fuente externa produjo una operacion mas estable.
Locomocion	Alinear patas, activar caminata continua y medir el recorrido sobre una superficie plana.	El robot avanzo aproximadamente 1.08 m con pasos, balanceo y desviacion ligera.
Interfaz	Abrir la GUI, revisar controles y comprobar sintaxis del programa Python.	La ventana inicia correctamente e incluye lectura serial no bloqueante, telemetria y registro.





6.1 Problemas y soluciones

Problema	Solucion implementada
Motores con diferente velocidad	Se probaron individualmente y el firmware final se adapto a un motor funcional.
Falla del puente H o motor	Se revisaron conexiones, pines y alimentacion; se simplifico la configuracion.
Pila de 9 V insuficiente	Se separo la alimentacion del motor y se mantuvo tierra comun.
Desviacion y piezas flojas	Se reforzaron uniones, se bajo el centro de gravedad y se ampliaron apoyos.
GUI sin recepcion de datos	Se agrego lectura periodica, indicador de distancia, desconexion y registro de telemetria.

7. Conclusiones

Conclusion grupal

El proyecto cumplio su objetivo principal: integrar un robot caminante sin ruedas con control de motor, deteccion de obstaculos, alerta sonora e interfaz de computadora. La arquitectura orientada a objetos hizo posible modificar el hardware sin rehacer toda la logica. Los resultados tambien mostraron que en robotica el codigo no puede evaluarse de forma aislada: la alimentacion, la alineacion y la resistencia mecanica determinan el comportamiento final.

Conclusiones

Diego De la O Arellano: Aprendi que la estabilidad depende del peso, el ancho de los pies y la posicion del centro de gravedad. El reto principal fue evitar que el robot perdiera equilibrio. Mejoraria los soportes y las uniones.

Hazel Ramirez Vazquez: Aprendi a relacionar el orden de las conexiones con el comportamiento mecanico. El reto fue mantener estable el montaje. Mejoraria la estructura del cuerpo y la simetria de las patas.

Diego Sebastian Espinoza Perez: Aprendi que los motores necesitan una fuente con corriente suficiente. El reto fue distinguir entre fallas del motor, del L293D y del cableado. Usaria una fuente estable desde las primeras pruebas.

Emilio Giovanni Meza Mendez: Aprendi a aplicar clases, interfaces y SOLID en un sistema fisico. El reto fue adaptar el programa al cambio de dos motores a uno. Mejoraria el diagnostico y la calibracion desde la interfaz.

David Alexander Torres Jalomo: Aprendi que la locomocion depende de alineacion, peso y resistencia de las uniones. El reto fue la desviacion durante la marcha. Mejoraria la simetria del mecanismo y los puntos de apoyo.

8. Referencias bibliograficas

- [1] Arduino, "UNO R3," Arduino Documentation. [En linea]. Disponible: <https://docs.arduino.cc/hardware/uno-rev3/>. [Consultado: 17-jun-2026].
- [2] Arduino, "Serial," Arduino Language Reference. [En linea]. Disponible: <https://docs.arduino.cc/language-reference/en/functions/communication/serial/>. [Consultado: 17-jun-2026].
- [3] STMicroelectronics, "L293D: Push-Pull Four Channel Driver with Diodes," hoja de datos. [En linea]. Disponible: <https://www.st.com/resource/en/datasheet/l293d.pdf>. [Consultado: 17-jun-2026].
- [4] Python Software Foundation, "tkinter - Python interface to Tcl/Tk," Python Documentation. [En linea]. Disponible: <https://docs.python.org/3/library/tkinter.html>. [Consultado: 17-jun-2026].
- [5] C. Liechti, "pySerial API," pySerial 3.5 Documentation. [En linea]. Disponible: https://pyserial.readthedocs.io/en/latest/pyserial_api.html. [Consultado: 17-jun-2026].
- [6] E. G. Meza Mendez et al., "Pinguino: Kagastian 5000," repositorio del proyecto. [En linea]. Disponible: <https://github.com/emiliogmeza-prog/Pinguino>. [Consultado: 17-jun-2026].

9. Anexos

Recurso	Ubicacion o enlace
Repositorio	https://github.com/emiliogmeza-prog/Pinguino
Video principal	programacion/video/proyecto.mp4
Demostracion del mecanismo	programacion/video/DemostracionMecanismo.mp4
Codigo Arduino	programacion/main/main.ino y archivos .h
Interfaz Python	programacion/python_gui/pinguino_gui_usb.py

Ejecutable	programacion/python_gui/dist/PinguinoRobotUSB.exe
Manual de usuario	programacion/docs/MANUAL_USUARIO.md
Manual tecnico	programacion/docs/MANUAL_TECNICO.md
Conexiones	programacion/docs/CONEXIONES.md
Diagramas y fotografias	programacion/imgs/

Anexo A. Flujo de operacion

- Conectar Arduino y la fuente externa del motor, comprobando que todas las tierras sean comunes.
- Cargar main/main.ino desde Arduino IDE y abrir la GUI de Python o el ejecutable.
- Seleccionar el puerto COM, presionar Conectar y verificar el indicador verde.
- Usar Automatico para operar con sensor o Caminata continua para una prueba mecanica.
- Observar distancia, mensajes y errores en el registro de telemetria.
- Presionar Detener y Desconectar antes de retirar la alimentacion.

Anexo B. Estructura del codigo

El sketch principal crea MotorL293D, UltrasonicSensor, BuzzerAlert y PenguinRobot. setup() inicia Serial a 9600 baudios y configura los dispositivos; loop() llama continuamente a robot.update(). La interfaz Python mantiene el ciclo de Tkinter y programa leer_serial() cada 100 ms mediante after(), evitando bloquear la ventana.

Anexo C. Configuracion de hardware

Pin Arduino	Dispositivo	Funcion
5	ENA del L293D	Salida PWM para velocidad.
2	IN1 del L293D	Sentido de giro del motor.
3	IN2 del L293D	Sentido de giro del motor.
8	TRIG del HC-SR04	Pulso de disparo ultrasonico.
9	ECHO del HC-SR04	Medicion del tiempo de retorno.
12	Buzzer	Salida de alerta sonora.

Anexo D. Lista de verificacion tecnica

- Confirmar que la fuente del motor no alimente directamente el pin de 5 V del Arduino.
- Unir GND de Arduino, L293D y fuente externa antes de activar el motor.
- Verificar que el mecanismo pueda moverse manualmente sin atorarse.
- Seleccionar Arduino UNO y el puerto COM correcto antes de cargar el firmware.
- Cerrar el Monitor Serial antes de conectar la interfaz Python al mismo puerto.
- Detener el robot inmediatamente si el L293D o el motor se calientan de forma anormal.